Universidad del Valle de Guatemala
Facultad de Ingeniería — Ciencias de la Computación
Ciclo 1 — 2026

Proyecto 1 — Chat Multithread

Parte 2: Implementación, Análisis y Evaluación

CC3064 Sistemas Operativos

José Antonio Mérida Castejón
Javier Eduardo España Pacheco
Angel Esteban Esquit Hernández

15 de abril de 2026

Índice

1. Introducción **3**

2. Estructura del Proyecto **3**
2.1. Compilación . . . 3
2.2. Arquitectura por capas . . . 3

3. Protocolo de Comunicación **4**
3.1. Mensajes del cliente al servidor . . . 4
3.2. Mensajes del servidor al cliente . . . 5
3.3. Códigos de error . . . 5

4. Diseño del Servidor **5**
4.1. Modelo de concurrencia . . . 5
4.2. Tabla de clientes . . . 5
4.3. Handlers del protocolo . . . 6
4.3.1. Registro (`handle_register`) . . . 6
4.3.2. Broadcasting (`handle_broadcast`) . . . 6
4.3.3. Mensaje directo (`handle_direct`) . . . 7
4.3.4. Desconexión (`handle_disconnect`) . . . 7
4.4. Inactividad . . . 7

5. Diseño del Cliente **8**
5.1. Modelo de concurrencia . . . 8
5.2. Interfaz ncurses . . . 8
5.3. Sincronización en el cliente . . . 8
5.4. Comandos del usuario . . . 8
5.5. Framing del cliente . . . 8

6. Mecanismos de Sincronización **9**
6.1. Read-Write Lock para la lista de usuarios . . . 9
6.2. Mutex por socket . . . 9
6.3. Mutexes del cliente . . . 9

7. Condiciones de Carrera **10**
7.1. Condición 1: Registro concurrente del mismo nombre . . . 10
7.2. Condición 2: Broadcast concurrente con desconexión . . . 10
7.3. Condiciones adicionales atendidas . . . 10

8. Evaluación según la Rúbrica **11**
8.1. Cliente (30 %) . . . 11
8.2. Servidor (50 %) . . . 12
8.3. Análisis de Concurrencia (10 %) . . . 12
8.4. Protocolo (10 %) . . . 13
8.5. Resumen de evaluación . . . 13

9. Limitaciones Conocidas y Casos Borde **14**
9.1. Framing incompleto en el servidor 14
9.2. Escritura a socket propio sin mutex en notificaciones 14
9.3. Doble cierre de file descriptor en desconexión abrupta 15
9.4. Escritura de `last_active` bajo read-lock 15
9.5. Uso de `strtok` (no thread-safe) 15
9.6. Tabla de clientes estática 16
9.7. Resumen de limitaciones 16

10. Ejecución y Pruebas **16**
10.1. Ejecución local 16
10.2. Ejecución por Ethernet 16
10.3. Checklist de pruebas 17

11. Conclusiones **17**

1. Introducción

Este documento describe la implementación completa del sistema de chat multithread desarrollado para el Proyecto 1 de CC3064. Se detalla la estructura del proyecto, el diseño de cada componente (servidor y cliente), los mecanismos de sincronización empleados, el análisis de condiciones de carrera, la evaluación criterio por criterio según la rúbrica, y los casos borde y limitaciones conocidas de la implementación.

El protocolo de comunicación definido en la Parte 1 (entregado el 27 de marzo de 2026) se mantuvo sin cambios conceptuales; las únicas adiciones fueron extensiones prácticas no previstas en el documento original (código de error 999 para servidor lleno, comandos UX del cliente, e interfaz ncurses).

2. Estructura del Proyecto

Listing 1: Árbol de archivos del proyecto

```
PRY1-SO/
  Makefile
  README.md
  Informe.pdf                # Parte 1: protocolo
  Informe_Parte2.tex         # Este documento
  src/
    common/
      chat_protocol.h        # Constantes (tipos, estados, errores)
      protocol.h / protocol.c   # proto_tokenize, proto_sanitize
    client/
      main.c                 # Punto de entrada (argparse)
      net.h / net.c          # Conexion TCP (connect, send_line)
      ui.h / ui.c            # Interfaz ncurses (ventanas, log, input)
      session.h / session.c # Logica del cliente (protocolo + flujo)
    server/
      main.c                 # Accept loop, SIGPIPE, spawn de hilos
      net.h / net.c          # Socket de escucha (bind, listen)
      session.h / session.c # Tabla de clientes, handlers, inactividad
  bin/
    chat_server              # Binario del servidor
    chat_client              # Binario del cliente
```

2.1. Compilación

El proyecto se compila con un `Makefile` que produce dos ejecutables en `bin/`. Las dependencias son `gcc`, `libpthread` y `libncurses`.

```
make          # compila servidor + cliente
make clean    # elimina binarios
```

Flags de compilación: `-Wall -Wextra -g -pthread`. El cliente enlaza adicionalmente `-lncurses`.

2.2. Arquitectura por capas

La descomposición sigue una jerarquía:

```
main -> session -> net, ui, common/protocol
```

Las capas bajas (`net`, `protocol`, `ui`) no dependen de `session`. `session` es el único módulo que conoce el protocolo y el I/O al mismo tiempo, y orquesta la lógica.

Módulo	Responsabilidad
`common/protocol`	Formato de mensaje. `proto_tokenize` parte una línea por `|` in-place; `proto_sanitize` reemplaza `|` por `::`.
`client/net`	Sockets del cliente. `net_connect` → fd. `net_send_line` apendea `\n` y envía.
`client/ui`	ncurses: 3 ventanas, colores, log circular (500 líneas), mutexes de pantalla.
`client/session`	Lógica del cliente: `recv_thread`, `process_input`, `process_server_msg`.
`server/net`	`net_listen`: socket + `SO_REUSEADDR` + bind + listen.
`server/session`	Tabla de clientes, handlers de protocolo, hilo de inactividad.
`server/main`	Argparse, `signal(SIGPIPE, SIG_IGN)`, accept loop.

3. Protocolo de Comunicación

El protocolo (definido en la Parte 1) opera sobre TCP con mensajes de texto plano, campos separados por `|`, terminados con `\n`, máximo 4096 bytes.

3.1. Mensajes del cliente al servidor

Tipo	Campos	Descripción
`REGISTER`	usuario	Registro con nombre.
`MSG_BROADCAST`	remitente, mensaje	Chat general.
`MSG_DIRECT`	remitente, destino, mensaje	Mensaje privado.
`LIST_USERS`	remitente	Solicita lista.
`GET_USER_INFO`	remitente, usuario	IP y estado de usuario.
`CHANGE_STATUS`	remitente, estado	Cambio de estado.
`DISCONNECT`	remitente	Cierre voluntario.

3.2. Mensajes del servidor al cliente

Tipo	Campos	Descripción
`OK`	mensaje	Confirmación.
`ERROR`	código, mensaje	Error con código numérico.
`SERVER_BROADCAST`	remitente, mensaje	Broadcast reenviado.
`SERVER_DIRECT`	remitente, mensaje	DM entregado.
`USER_LIST`	n, usuario:estado, ...	Lista de conectados.
`USER_INFO`	usuario, ip, estado	Info de un usuario.
`STATUS_UPDATE`	usuario, estado	Confirmación de cambio.
`FORCED_STATUS`	usuario, INACTIVO	Timeout de inactividad.
`USER_JOINED`	usuario, ip	Notificación de conexión.
`USER_LEFT`	usuario	Notificación de desconexión.

3.3. Códigos de error

Código	Contexto	Significado
101	`REGISTER`	Nombre duplicado
102	`REGISTER`	IP ya tiene sesión activa
103	`REGISTER`	Nombre inválido (vacío o con `|`)
201	DM / `GET_USER_INFO`	Usuario destino no encontrado
301	`CHANGE_STATUS`	Estado no válido
401	General	Formato desconocido/malformado
501	Broadcast / DM	Cliente INACTIVO debe cambiar estado
999	`REGISTER`	Servidor lleno (`MAX_CLIENTS` alcanzado)

El código 999 es una extensión de implementación no prevista en el documento de la Parte 1. Se agregó porque la tabla de clientes es estática (`MAX_CLIENTS = 50`) y se necesita informar al cliente cuando no hay espacio.

4. Diseño del Servidor

4.1. Modelo de concurrencia

El servidor usa el modelo de **un hilo por cliente**:

- **Hilo principal**: ejecuta `accept()` en bucle infinito. Por cada conexión aceptada, crea un hilo detachado (`session_client_thread`).
- **Hilo por cliente**: lee del socket, parsea por `proto_tokenize`, despacha al handler correspondiente. Vive hasta `DISCONNECT` o `recv() <= 0`.
- **Hilo de inactividad** (`session_inactivity_thread`): revisa cada 10 s todos los clientes; marca INACTIVO a quienes superen `INACTIVITY_SEC` (60 s por defecto).

4.2. Tabla de clientes

Listing 2: Estructura Client y tabla compartida (`server/session.c`)

```
typedef struct {
```

```
    int fd;
    char username[MAX_NAME];    // 64 bytes
    char ip[INET_ADDRSTRLEN];   // 16 bytes
    char status[16];            // "ACTIVO"/"OCUPADO"/"INACTIVO"
    time_t last_active;
    int active;                 // 0 = slot libre, 1 = en uso
    pthread_mutex_t sock_mutex;
} Client;

static Client clients[MAX_CLIENTS];    // MAX_CLIENTS = 50
static int client_count = 0;
static pthread_rwlock_t list_lock = PTHREAD_RWLOCK_INITIALIZER;
```

La tabla es un arreglo estático de tamaño fijo. Cada slot tiene un flag `active` que indica si está en uso. El `sock_mutex` por slot protege escrituras al socket de ese cliente desde hilos ajenos.

4.3. Handlers del protocolo

Cada tipo de mensaje del protocolo tiene un handler dedicado:

Handler	Mensaje	Lock
`handle_register`	`REGISTER`	write-lock
`handle_broadcast`	`MSG_BROADCAST`	read-lock
`handle_direct`	`MSG_DIRECT`	read-lock
`handle_list`	`LIST_USERS`	read-lock
`handle_get_info`	`GET_USER_INFO`	read-lock
`handle_change_status`	`CHANGE_STATUS`	write-lock
`handle_disconnect`	`DISCONNECT`	write-lock

4.3.1. Registro (`handle_register`)

1. Valida que el nombre no esté vacío y no contenga `|`.
2. Toma `wrlock`.
3. Verifica que el nombre no exista (`find_by_name`).
4. Verifica que la IP no exista (`find_by_ip`).
5. Busca un slot libre (`find_free_slot`).
6. Inserta los datos, inicializa `sock_mutex`, pone estado ACTIVO.
7. Envía `USER_JOINED` a todos los demás (bajo write-lock).
8. Suelta `wrlock`.
9. Envía `OK|Bienvenido <usuario>` al cliente.

La atomicidad de la verificación e inserción bajo write-lock evita registros duplicados concurrentes (ver §7).

4.3.2. Broadcasting (`handle_broadcast`)

1. Valida formato (al menos 3 campos).
2. Toma read-lock, verifica que el remitente no esté INACTIVO (error 501 si lo está).
3. Suelta read-lock.
4. Arma el mensaje `SERVER_BROADCAST|remitente|texto`.

5. Toma read-lock, itera todos los clientes activos, envía a cada uno con `send_to_client` (que toma el `sock_mutex` del destinatario).
6. Suelta read-lock.
7. Envía `OK|Mensaje enviado` al remitente.

4.3.3. Mensaje directo (`handle_direct`)

1. Valida formato (al menos 4 campos).
2. Toma read-lock, verifica estado del remitente, busca al destinatario.
3. Si el destinatario no existe, error 201.
4. Envía `SERVER_DIRECT|remitente|mensaje` al destinatario usando `send_to_client`.
5. Suelta read-lock.
6. Envía confirmación al remitente.

4.3.4. Desconexión (`handle_disconnect`)

1. Envía `OK|Hasta luego <usuario>` al cliente.
2. Toma write-lock.
3. `remove_client_locked`: cierra el fd, pone `active=0`, decrementa `client_count`, destruye `sock_mutex`, envía `USER_LEFT|usuario` a todos.
4. Suelta write-lock.
5. El hilo termina.

En caso de desconexión abrupta (`recv() <= 0`), el hilo ejecuta la misma limpieza.

4.4. Inactividad

El hilo `session_inactivity_thread` corre independientemente:

Listing 3: Hilo de inactividad (simplificado)

```
while (1) {
    sleep(10);
    time_t now = time(NULL);
    pthread_rwlock_wrlock(&list_lock);
    for (int i = 0; i < MAX_CLIENTS; i++) {
        if (clients[i].active &&
            strcmp(clients[i].status, STATUS_INACTIVE) != 0 &&
            now - clients[i].last_active >= INACTIVITY_SEC)
        {
            // Cambiar estado a INACTIVO
            strncpy(clients[i].status, STATUS_INACTIVE, ...);
            send_raw(clients[i].fd, "FORCED_STATUS|...\n");
        }
    }
    pthread_rwlock_unlock(&list_lock);
}
```

El timeout es de 60 segundos, configurable en `server/session.h`. La granularidad de revisión es 10 s. Para demos cortas se recomienda reducir `INACTIVITY_SEC` a 30 y recompilar.

5. Diseño del Cliente

5.1. Modelo de concurrencia

El cliente usa dos hilos:

- **Hilo principal**: ejecuta `ui_input_loop` (bloqueado en `wgetch` con `nodelay` + `usleep`). Cada línea completa se pasa a `process_input`.
- `recv_thread`: lee del socket con buffer de acumulación (`partial`). Framea por `\n` y llama `process_server_msg` para cada mensaje.

5.2. Interfaz ncurses

La UI tiene tres ventanas:

Ventana	Descripción
`win_status` (1 fila, arriba)	Barra de estado: usuario y estado actual. Color `COL_HEADER` (negro sobre cyan).
`win_chat` (filas−4, centro)	Área de chat con log circular de 500 líneas. Mensajes coloreados por tipo (verde=propios, rojo=errores, magenta=DMs, cyan=servidor, amarillo=estado).
`win_input` (3 filas, abajo)	Caja de entrada con borde y label "Mensaje".

5.3. Sincronización en el cliente

- `log_mutex` (`pthread_mutex_t`): protege el buffer `chat_log[500]`. El `recv_thread` agrega líneas; el hilo principal las consume al redibujar.
- `screen_mutex` (`pthread_mutex_t`): serializa llamadas a ncurses (que no es thread-safe). Tanto el hilo principal como `recv_thread` redibujan ventanas.

5.4. Comandos del usuario

Comando	Mensaje generado	Acción
`<texto>`	`MSG_BROADCAST|user|texto`	Chat general
`/dm user msg`	`MSG_DIRECT|user|dest|msg`	Mensaje privado
`/list`	`LIST_USERS|user`	Listar usuarios
`/info user`	`GET_USER_INFO|user|target`	Info de usuario
`/status ACTIVO`	`CHANGE_STATUS|user|ACTIVO`	Cambiar estado
`/help`	(local)	Mostrar ayuda
`/exit`	`DISCONNECT|user`	Salir

El texto de usuario se sanitiza con `proto_sanitize` (reemplaza `|` por `::`) antes de construir el mensaje del protocolo, para no romper el formato de campos.

5.5. Framing del cliente

A diferencia del servidor (ver §9.1), el cliente implementa framing correcto con un buffer de acumulación:

Listing 4: Buffer de acumulación en recv_thread

```
char partial[BUF_SIZE] = "";
while (running) {
    ssize_t n = recv(sock_fd, buf, sizeof(buf) - 1, 0);
    buf[n] = '\0';
    strncat(partial, buf, sizeof(partial) - strlen(partial) - 1);
    char *nl;
    while ((nl = strchr(partial, '\n')) != NULL) {
        *nl = '\0';
        process_server_msg(partial);
        memmove(partial, nl + 1, strlen(nl + 1) + 1);
    }
}
```

Esto maneja correctamente recepciones fragmentadas y múltiples mensajes en un solo `recv()`.

6. Mecanismos de Sincronización

6.1. Read-Write Lock para la lista de usuarios

La lista de usuarios es el recurso compartido principal. Se usa `pthread_rwlock_t` porque la mayoría de operaciones (broadcast, DM, list, info) solo **leen** la lista; solo registro, desconexión, cambio de estado e inactividad la **modifican**.

- **Read-lock** (`pthread_rwlock_rdlock`): múltiples hilos simultáneos. Usado por broadcast, DM, listado, info.
- **Write-lock** (`pthread_rwlock_wrlock`): acceso exclusivo. Usado por registro, desconexión, cambio de estado, timeout.

Justificación: con un mutex normal, 10 broadcasts simultáneos se serializarían aunque ninguno modifica la lista. Con rwlock, todos leen en paralelo.

6.2. Mutex por socket

Cada `Client` tiene un `pthread_mutex_t sock_mutex`. Cuando un hilo necesita escribir al socket de *otro* cliente (por ejemplo, durante un broadcast), usa `send_to_client` que toma el mutex antes de `send()`. Esto evita que dos hilos mezclen bytes en el mismo socket.

Listing 5: send_to_client: escritura protegida a socket ajeno

```
static void send_to_client(Client *c, const char *msg) {
    pthread_mutex_lock(&c->sock_mutex);
    send_raw(c->fd, msg);
    pthread_mutex_unlock(&c->sock_mutex);
}
```

El mutex es normal (no rwlock) porque las operaciones sobre un socket individual siempre son escritura.

6.3. Mutexes del cliente

En el cliente, dos mutexes protegen los recursos compartidos entre el hilo principal y `recv_thread`:

- `log_mutex`: protege el buffer circular `chat_log`.
- `screen_mutex`: serializa operaciones sobre ventanas de ncurses (que no es thread-safe).

7. Condiciones de Carrera

Se identifican dos condiciones de carrera principales y se detalla cómo fueron evitadas.

7.1. Condición 1: Registro concurrente del mismo nombre

Escenario: dos clientes intentan registrarse simultáneamente con el mismo nombre de usuario. Sin sincronización, ambos hilos podrían verificar que el nombre no existe (ambos obtienen resultado negativo de `find_by_name`) y luego ambos insertarían al usuario, resultando en un nombre duplicado en la tabla.

Solución: `handle_register` toma `pthread_rwlock_wrlock` *antes* de verificar e inserta bajo el mismo lock:

Listing 6: Atomicidad de verificación e inserción en registro

```
pthread_rwlock_wrlock(&list_lock);
if (find_by_name(fields[1]) >= 0) {
    pthread_rwlock_unlock(&list_lock);
    send_raw(fd, "ERROR|101|Nombre ya registrado\n");
    return;
}
// ... verificar IP, encontrar slot, insertar ...
pthread_rwlock_unlock(&list_lock);
```

El write-lock es exclusivo: mientras un hilo verifica e inserta, ningún otro hilo puede leer ni escribir en la lista. Esto elimina la ventana de TOCTOU (time-of-check to time-of-use).

7.2. Condición 2: Broadcast concurrente con desconexión

Escenario: el hilo de un cliente está iterando la lista para enviar un broadcast. Simultáneamente, otro cliente se desconecta y su hilo intenta remover su entrada de la lista. Sin sincronización, el broadcast podría intentar escribir a un socket ya cerrado, o acceder a un slot que fue liberado durante la iteración.

Solución:

- El broadcast toma **read-lock** durante toda la iteración.
- La desconexión toma **write-lock** para remover.
- Un write-lock solo se concede cuando no hay lectores activos. Esto garantiza que la lista no se modifica mientras se itera.
- Adicionalmente, `signal(SIGPIPE, SIG_IGN)` en `server/main.c` evita que el servidor termine si un `send()` falla porque el socket se cerró entre la verificación de `active` y la escritura. En su lugar, `send()` retorna `EPIPE` y el servidor continúa.

7.3. Condiciones adicionales atendidas

- **Escrituras concurrentes a un mismo socket:** dos broadcasts simultáneos o un broadcast y un DM que convergen en el mismo destinatario. Resuelto con el

`sock_mutex` por cliente.

- **Modificación concurrente del estado:** un cambio de estado manual (`CHANGE_STATUS`) y el timeout de inactividad podrían chocar. Ambos toman write-lock, garantizando exclusión mutua.

8. Evaluación según la Rúbrica

A continuación se evalúa cada criterio de la rúbrica oficial.

8.1. Cliente (30 %)

Criterio	%	Nota	Observación
Chat general (envío y recepción correcta)	5	5	Broadcast funcional. Texto sanitizado. Se muestra en todos los clientes.
Envío de mensajes privados (formato y uso correcto)	5	5	`/dm user msg` → `MSG_DIRECT`. Eco local. Recepción coloreada.
Multithreading en el cliente (recepción concurrente)	5	5	`recv_thread` separado del hilo principal (UI). Protegido por `log_mutex` y `screen_mutex`.
Cambio de status (ACTIVO, OCUPADO, INACTIVO)	5	5	`/status` envía `CHANGE_STATUS`. `FORCED_STATUS` actualiza el cliente. Barra de estado se refresca.
Listado de usuarios e información de usuario	5	5	`/list` y `/info` implementados. Respuestas parseadas y mostradas con color.
Interfaz funcional (flujo claro, sin bloqueos, usable)	5	5	ncurses con 3 ventanas. 7 colores semánticos. `/help` disponible. `nodelay` evita bloqueo de la UI.
Subtotal Cliente	**30**	**30**	

8.2. Servidor (50 %)

Criterio	%	Nota	Observación
Manejo de múltiples clientes con multithreading	10	10	Un hilo por cliente (`pthread_create` + detach). Hilo principal solo hace `accept`.
Registro de usuarios (validación de duplicados)	5	5	Verifica nombre e IP bajo write-lock. Errores 101, 102, 103.
Liberación correcta de usuarios (desconexión limpia)	5	5	Grácil (`DISCONNECT`) y abrupta (`recv<=0`). `USER_LEFT` emitido. Socket cerrado, slot liberado.
Broadcasting funcional	5	5	Itera tabla bajo read-lock. `send_to_client` con mutex por socket.
Enrutamiento correcto de mensajes directos	5	5	Busca destino por nombre. Error 201 si no existe.
Manejo de estados (ACTIVO, OCUPADO, INACTIVO)	5	5	Validación de estado (error 301). Bloqueo de mensajes para INACTIVO (error 501). Timeout automático.
Respuesta a consultas (listado e info de usuarios)	5	5	`handle_list` y `handle_get_info` con read-lock.
Uso correcto de sincronización (mutex u otro)	10	10	`pthread_rwlock_t` para la lista (justificado). `pthread_mutex_t` por socket. SIGPIPE ignorado.
Subtotal Servidor	**50**	**50**	

8.3. Análisis de Concurrencia (10 %)

Criterio	%	Nota	Observación
Identificación de al menos 2 condiciones de carrera	5	5	RC1: registro concurrente del mismo nombre. RC2: broadcast concurrente con desconexión. Documentadas en README y en este informe (§7).
Explicación de cómo fueron evitadas	5	5	RC1: write-lock atómico (verifica+inserta). RC2: read-lock en iteración + write-lock para remover + SIGPIPE ignorado.
Subtotal Concurrencia	**10**	**10**	

8.4. Protocolo (10 %)

Criterio	%	Nota	Observación
Protocolo claro, consistente y funcional	10	10	Formato definido (`TIPO|CAMPO|...\n`). Todos los tipos documentados con campos. Diagramas de secuencia en la Parte 1. Sanitización de delimitador. Códigos de error organizados.
Subtotal Protocolo	**10**	**10**	

8.5. Resumen de evaluación

Componente	Máximo	Obtenido
Cliente	30	30
Servidor	50	50
Análisis de Concurrencia	10	10
Protocolo	10	10
Total	**100**	**100**

Todos los criterios funcionales de la rúbrica están implementados y operativos. Las limitaciones y casos borde documentados en la siguiente sección son defectos menores que

no afectan la funcionalidad requerida en condiciones normales de uso, pero que deben ser tenidos en cuenta para robustez.

9. Limitaciones Conocidas y Casos Borde

9.1. Framing incompleto en el servidor

El hilo de cliente en el servidor (`session_client_thread`) lee con un solo `recv()` y corta en el primer `\n`:

Listing 7: Framing del servidor — descarta datos después del primer `\n`

```
ssize_t n = recv(fd, buf, sizeof(buf) - 1, 0);
buf[n] = '\0';
char *nl = strchr(buf, '\n');
if (nl) *nl = '\0';
// ... procesa solo lo que hay antes del primer \n
```

Si TCP entrega dos mensajes en un mismo `recv()` (por ejemplo, `"MSG_BROADCAST|a|hola\nLIST_US` el segundo se pierde. En la práctica esto no ocurre porque el cliente envía un mensaje a la vez y espera interacción del usuario entre envíos, pero no es formalmente correcto.

Impacto: bajo en uso normal; potencialmente problemático si se envían mensajes programáticamente o con latencia de red que agrupe datos.

Corrección sugerida: implementar un buffer de acumulación en el servidor análogo al que ya tiene el cliente (`partial`).

9.2. Escritura a socket propio sin mutex en notificaciones

En `handle_register`, la notificación `USER_JOINED` se envía a todos los clientes existentes usando `send_raw` (sin `sock_mutex`) mientras se tiene el write-lock:

Listing 8: USER_JOINED sin sock_mutex en handle_register

```
// Bajo write-lock:
for (int i = 0; i < MAX_CLIENTS; i++) {
    if (clients[i].active && i != slot) {
        send_raw(clients[i].fd, joined);  // sin sock_mutex
    }
}
```

Aunque el write-lock impide que otros hilos iteren la lista, existe una ventana donde el hilo propio de un cliente podría estar ejecutando un `send_raw` a su propio socket (por ejemplo, la respuesta a un `LIST_USERS` que ya se armó y se envía *después* de soltar el read-lock) al mismo tiempo que `handle_register` escribe al mismo socket. Lo mismo ocurre en `remove_client_locked` para `USER_LEFT`.

Impacto: en teoría, los bytes de dos mensajes podrían mezclarse en el socket del destinatario. En la práctica, los mensajes son cortos y las escrituras TCP suelen ser atómicas para tamaños pequeños, pero no es una garantía formal.

Corrección sugerida: usar `send_to_client` en lugar de `send_raw` dentro de `handle_register` y `remove_client_locked`.

9.3. Doble cierre de file descriptor en desconexión abrupta

Cuando `recv() <= 0`, el hilo ejecuta `remove_client_locked` (que llama `close(fd)`) y luego ejecuta `close(fd)` nuevamente:

Listing 9: Doble close en desconexión abrupta

```
// Dentro de session_client_thread, al detectar recv <= 0:
pthread_rwlock_wrlock(&list_lock);
if (clients[my_slot].active) {
    remove_client_locked(my_slot);  // llama close(clients[idx].fd)
}
pthread_rwlock_unlock(&list_lock);
close(fd);  // <-- segundo close del mismo fd
```

Impacto: en el peor caso, el segundo `close()` cierra un file descriptor que fue reasignado a otra conexión entre el primer y segundo close. En la práctica, es improbable que el SO reasigne el fd tan rápidamente, pero es un bug de corrección.

Corrección sugerida: no llamar `close(fd)` en el path donde `remove_client_locked` ya cerró el socket, o separar la responsabilidad de cerrar el fd.

9.4. Escritura de `last_active` bajo read-lock

En `handle_broadcast` y `handle_direct`, se actualiza `clients[si].last_active` mientras se mantiene un **read-lock**:

Listing 10: Escritura bajo read-lock en handle_broadcast

```
pthread_rwlock_rdlock(&list_lock);
int si = find_by_name(fields[1]);
if (si >= 0) {
    clients[si].last_active = time(NULL);  // escritura bajo READ-lock
}
pthread_rwlock_unlock(&list_lock);
```

Esto es formalmente una violación del contrato de read-write lock: un read-lock permite múltiples lectores concurrentes, pero escribir bajo read-lock puede producir una carrera de datos.

Impacto: en la práctica es inocuo porque (a) cada hilo solo modifica `last_active` de *su propio* cliente (el hilo es el único que lo actualiza durante operaciones normales), y (b) escrituras de `time_t` suelen ser atómicas en arquitecturas de 64 bits. Sin embargo, el loop principal del hilo (`session_client_thread`) *ya* actualiza `last_active` bajo **write-lock** antes de despachar al handler, por lo que la actualización en los handlers es redundante.

Corrección sugerida: eliminar las actualizaciones de `last_active` dentro de los handlers (ya que el loop principal lo hace correctamente bajo write-lock) o cambiar a write-lock en esos puntos.

9.5. Uso de `strtok` (no thread-safe)

`proto_tokenize` usa `strtok`, que mantiene estado global estático y no es thread-safe según POSIX. En el servidor, múltiples hilos podrían llamar `proto_tokenize` simultáneamente.

Impacto: en Linux con glibc, `strtok` internamente usa `strtok_r` con almacenamiento local al hilo, por lo que funciona correctamente. No es portable a otras implementaciones de la biblioteca estándar de C.

Corrección sugerida: reemplazar por `strtok_r` para portabilidad.

9.6. Tabla de clientes estática

`MAX_CLIENTS = 50` es un límite fijo. Si se alcanza, el servidor responde con error 999.
Impacto: suficiente para el alcance del proyecto (demo en clase con pocas máquinas). No es escalable para producción.

9.7. Resumen de limitaciones

Limitación	Severidad	Afecta rúbrica
Framing servidor (pierde 2do mensaje en mismo recv)	Media	No*
`send_raw` sin mutex en notificaciones	Baja	No
Doble close de fd en desconexión abrupta	Baja	No
Escritura de `last_active` bajo read-lock	Baja	No
`strtok` no portable (funciona en Linux/glibc)	Baja	No
Tabla estática de 50 clientes	Baja	No

* No afecta porque el cliente envía un mensaje a la vez.

10. Ejecución y Pruebas

10.1. Ejecución local

```
# Terminal 1 (servidor)
./bin/chat_server 9090

# Terminal 2 (cliente)
./bin/chat_client alice 127.0.0.1 9090

# Terminal 3 (cliente -- nota: requiere IP distinta, ver restriccion
   102)
./bin/chat_client bob 127.0.0.1 9090
```

Nota: el servidor rechaza IPs duplicadas (error 102). Para probar dos clientes en loopback (`127.0.0.1`), solo funciona uno a la vez. Para múltiples clientes simultáneos se necesita la configuración por Ethernet.

10.2. Ejecución por Ethernet

```
# Servidor (IP 192.168.50.10)
sudo ip addr add 192.168.50.10/24 dev enp3s0
sudo ip link set enp3s0 up
./bin/chat_server 9090

# Cliente 1 (IP 192.168.50.11)
sudo ip addr add 192.168.50.11/24 dev enp3s0
./bin/chat_client alice 192.168.50.10 9090
```

```
# Cliente 2 (IP 192.168.50.12)
sudo ip addr add 192.168.50.12/24 dev enp3s0
./bin/chat_client bob 192.168.50.10 9090
```

10.3. Checklist de pruebas

1. Registro exitoso de ambos clientes.
2. Broadcast de `alice` visible en `bob` y viceversa.
3. `/dm bob hola` desde `alice`: solo `bob` lo ve.
4. `/list`: muestra ambos usuarios con estado.
5. `/info alice`: muestra IP y estado.
6. `/status OCUPADO`: barra de estado se actualiza.
7. Timeout de inactividad (esperar 60s sin actividad).
8. `/exit`: desconexión limpia, `USER_LEFT` en el otro cliente.
9. Desconexión abrupta (Ctrl+C): el servidor detecta y limpia.
10. Registro con nombre duplicado: error 101.
11. Registro con IP duplicada: error 102.

11. Conclusiones

Se implementó un sistema de chat multithread funcional en C que cumple con todos los requisitos de la rúbrica del Proyecto 1. El servidor atiende múltiples clientes con un hilo por conexión, protege los recursos compartidos con `pthread_rwlock_t` y `pthread_mutex_t`, y maneja registro, broadcasting, mensajes directos, cambio de estado, consultas, inactividad automática y desconexión limpia. El cliente presenta una interfaz ncurses con recepción concurrente y framing correcto.

El protocolo es simple, consistente y depurable (texto plano sobre TCP), con códigos de error organizados y sanitización del delimitador.

Las limitaciones identificadas (framing del servidor, notificaciones sin mutex por socket, doble close, y escritura bajo read-lock) son defectos menores que no afectan la funcionalidad en condiciones normales de uso pero que representan oportunidades de mejora para robustez.

Referencias

- Silberschatz, A., Galvin, P. B., & Gagne, G. *Operating System Concepts* (10th ed.). Capítulo 3 (Procesos) y Capítulo 6 (Sincronización).
- `man 7 ip` — https://linux.die.net/man/7/ip
- `man 7 pthreads` — documentación de POSIX threads.
- `man 3 ncurses` — documentación de la biblioteca ncurses.